Cryptography & Information Security

Practical File

PRACTICAL NO. 1(A) : CAESAR CIPHER

AIM: To implement encryption and decryption of a message using the Caesar (shift) cipher.

THEORY: The Caesar cipher is a substitution cipher: each letter is replaced by the letter k positions ahead in the alphabet, wrapping from Z back to A. Encryption is E(x) = (x + k) mod 26 and decryption is D(x) = (x − k) mod 26, where x is the letter index (A = 0, ..., Z = 25); digits and punctuation pass through unchanged. With only 25 meaningful keys it falls to brute force or frequency analysis, so it survives as the standard introduction to substitution and modular arithmetic.

PROGRAM:

```
def encrypt(text, k):
    out = ""
    for ch in text:
        if ch.isupper():
            out += chr((ord(ch) - 65 + k) % 26 + 65)
        elif ch.islower():
            out += chr((ord(ch) - 97 + k) % 26 + 97)
        else:
            out += ch
    return out

msg = input("Enter Message: ")
k = int(input("Enter Shift Value: "))
enc = encrypt(msg, k)
print("Encrypted Message:", enc)
print("Decrypted Message:", encrypt(enc, -k))
```

RESULT: With shift 3 the message "Attack at Dawn" encrypted to "Dwwdfn dw Gdzq" and decrypted back to the original plaintext.

PRACTICAL NO. 1(B) : RAIL FENCE CIPHER

AIM: To encrypt a message using the rail fence (zigzag) transposition technique.

THEORY: The rail fence is a transposition cipher: letters keep their identity but change position. The plaintext is written diagonally downwards and upwards across r rails and the ciphertext is read off row by row; the key is simply r, and the zigzag repeats with a period of 2r − 2. Since no letter is changed, the ciphertext preserves the plaintext letter frequencies — the classic giveaway that a transposition, not a substitution, was applied.

PROGRAM:

```
def encrypt(text, rails):
    fence = [''] * rails
    row, step = 0, 1
    for ch in text:
        fence[row] += ch
        if row == 0:
            step = 1
        elif row == rails - 1:
            step = -1
        row += step
    return ''.join(fence)

msg = input("Enter Message: ")
rails = int(input("Enter Rails: "))
print("Encrypted Message:", encrypt(msg, rails))
```

RESULT: "Meet Me After The Toga Party" over 3 rails read back as "M AregaetM fe h oaPryeetTT t" — same letters, different order.

PRACTICAL NO. 2 : HMAC - MESSAGE AUTHENTICATION

AIM: To generate a Hash-based Message Authentication Code (HMAC) at the sender and verify it at the receiver using a shared secret key.

THEORY: HMAC binds a hash (here SHA-256) to a secret key: HMAC(K, M) = H((K XOR opad) || H((K XOR ipad) || M)), where ipad = 0x36 and opad = 0x5c are repeated to the 64-byte block size. A plain hash can be recomputed by anyone, but an attacker who edits the message cannot produce a matching tag without the key. The receiver recomputes the tag and compares it with hmac.compare_digest(), a constant-time comparison that blocks timing attacks.

PROGRAM:

Sender:

```
import hmac, hashlib
msg = input("Enter Message: ")
key = input("Enter Secret Key: ")
mac = hmac.new(key.encode(), msg.encode(), hashlib.sha256).hexdigest()
print("Generated MAC:", mac)
```

Receiver:

```
import hmac, hashlib
msg = input("Enter Received Message: ")
key = input("Enter Secret Key: ")
rx_mac = input("Enter Received MAC: ")
mac = hmac.new(key.encode(), msg.encode(), hashlib.sha256).hexdigest()
if hmac.compare_digest(mac, rx_mac):
    print("Message is Authentic.")
else:
    print("Message Authentication Failed!")
```

RESULT: The receiver recomputed the MAC over the received message and it matched the transmitted tag — "Message is Authentic".

PRACTICAL NO. 3 : DIGITAL SIGNATURE (RSA-PSS)

AIM: To sign a message with the sender's RSA private key and verify the signature at the receiver using the corresponding public key.

THEORY: The sender hashes the message with SHA-256 and signs the hash using RSA-PSS (Probabilistic Signature Scheme), where a random salt makes every signature different. Anyone holding the public key (exported as PEM) can verify, but nobody without the private key can forge — a match proves integrity, authenticity and non-repudiation. Signing the hash instead of the whole message keeps the operation fast and bounds the signature size to the RSA modulus.

PROGRAM:

Sender:

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization

key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
msg = b"Hello Receiver"
sig = key.sign(msg, padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH), hashes.SHA256())
open("message.txt", "wb").write(msg)
open("signature.sig", "wb").write(sig)
open("public_key.pem", "wb").write(key.public_key().public_bytes(
    serialization.Encoding.PEM,
    serialization.PublicFormat.SubjectPublicKeyInfo))
print("Message and Signature Sent")
```

Receiver:

```
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes, serialization

key = serialization.load_pem_public_key(open("public_key.pem", "rb").read())
msg = open("message.txt", "rb").read()
sig = open("signature.sig", "rb").read()
try:
    key.verify(sig, msg, padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
               salt_length=padding.PSS.MAX_LENGTH), hashes.SHA256())
    print("Signature Verified")
    print("Message is Authentic")
except:
    print("Signature Verification Failed")
```

RESULT: A 2048-bit key pair was generated and "Hello Receiver" was signed with RSA-PSS + SHA-256; the receiver reported "Signature Verified".

PRACTICAL NO. 4 : RSA ENCRYPTION

AIM: To implement RSA public-key encryption and decryption using small primes.

THEORY: Key generation: choose primes p, q and compute $n = p \cdot q$ and $\varphi(n) = (p - 1)(q - 1)$; pick e coprime to $\varphi(n)$ and derive the private exponent $d = e^{-1} \bmod \varphi(n)$. Encryption is $C = M^e \bmod n$ and decryption is $M = C^d \bmod n$, valid because $e \cdot d \equiv 1 \pmod{\varphi}$ lets Euler's theorem collapse the exponent back to M. With p = 11, q = 13: n = 143, $\varphi = 120$, e = 7, d = 103. Security rests on the difficulty of factoring n — real systems use 2048-bit-plus moduli and encrypt only a session key.

PROGRAM:

Sender:

```
import math
p = int(input("Enter p: "))
q = int(input("Enter q: "))
msg = input("Enter message: ")
n, phi = p * q, (p - 1) * (q - 1)
e = next(k for k in range(2, phi) if math.gcd(k, phi) == 1)
d = pow(e, -1, phi)
cipher = [str(pow(ord(c), e, n)) for c in msg]
print("Public Key:", e, n)
print("Private Key:", d, n)
print("Cipher Text:", " ".join(cipher))
```

Receiver:

```
d = int(input("Enter d: "))
n = int(input("Enter n: "))
cipher = input("Enter cipher text: ")
msg = "".join(chr(pow(int(c), d, n)) for c in cipher.split())
print("Decrypted Message:", msg)
```

RESULT: With p = 11, q = 13 the message "Hi" encrypted to the ciphertext "19 118" and decrypted back to "Hi".

PRACTICAL NO. 5 : DIFFIE-HELLMAN KEY EXCHANGE

AIM: To establish a shared secret key between two parties communicating over an insecure channel using the Diffie-Hellman key exchange.

THEORY: With public prime p and primitive root g, the sender picks secret a and sends $A = g^a \bmod p$; the receiver picks secret b and sends $B = g^b \bmod p$. Both then compute the same key $s = B^a \bmod p = A^b \bmod p = g^{ab} \bmod p$. With p = 23, g = 5, a = 6, b = 15: A = 8, B = 19, s = 2. An eavesdropper must solve the discrete logarithm problem to recover the secret; plain DH authenticates nobody, so real protocols sign the exchange to stop man-in-the-middle.

PROGRAM:

Sender:

```
p = int(input("Enter prime number p: "))
g = int(input("Enter primitive root g: "))
a = int(input("Enter Sender private key: "))
A = pow(g, a, p)
print("Sender Public Key:", A)
B = int(input("Enter Receiver Public Key: "))
print("Sender Shared Key:", pow(B, a, p))
```

Receiver:

```
p = int(input("Enter prime number p: "))
g = int(input("Enter primitive root g: "))
b = int(input("Enter Receiver private key: "))
B = pow(g, b, p)
print("Receiver Public Key:", B)
A = int(input("Enter Sender Public Key: "))
print("Receiver Shared Key:", pow(A, b, p))
```

RESULT: Both parties derived the identical shared key 2 without ever transmitting it.

PRACTICAL NO. 6 : MALWARE DETECTION USING VIRUSTOTAL

AIM: To analyze a suspicious file using VirusTotal and identify malware detection results and file properties.

REQUIREMENTS:

Computer or laptop with internet connection

VirusTotal (virustotal.com) account

Authorized suspicious file sample

THEORY: VirusTotal scans one sample against 70+ antivirus engines and reports a detection ratio (for example 45/72) with a threat label and category (trojan, worm, ransomware). Samples are identified by hashes — an identical file always yields the identical SHA-256, so analysts compare hashes first and upload only unknowns. PE analysis inspects the Windows executable structure (sections, entry point, timestamp) and TrID flags packers, since packed binaries that hide their real code are themselves a red flag.

PROCEDURE:

Select an authorized Windows executable sample.

Upload it to VirusTotal and record the SHA-256 hash.

Note the detection ratio, threat label and category.

Record file type, size, hashes and creation time.

Check PE details: target machine, sections, entry point.

Check TrID results for packing clues.

RESULT: The file's detection ratio, hashes, properties and PE/packing details were recorded from the VirusTotal report.

PRACTICAL NO. 7 : FIREWALL CONFIGURATION

AIM: To configure inbound and outbound firewall rules to block unauthorized access.

REQUIREMENTS:

Computer/Laptop with Windows OS

Windows Defender Firewall with Advanced Security

Web browser and Command Prompt

THEORY: Windows Defender Firewall is a stateful host firewall that filters traffic by port, program, protocol and IP address; each rule allows or blocks a connection under the Domain, Private and Public profiles. Web traffic uses TCP ports 80 and 443, so a port rule covering both stops browsing; a website is blocked by resolving it with nslookup and blocking its IP, and a program rule binds the block to the .exe so the app is silenced on any port.

PROCEDURE:

Block a Port (80, 443): Firewall → Advanced Settings → New Rule → Port → TCP → 80, 443 → Block the Connection → all profiles → Finish.

Block a Website: Run nslookup chatgpt.com, note the IP, create a Custom rule blocking that IP on all profiles.

Block a Program: New Rule → Program → browse to the .exe → Block the Connection → all profiles → Finish.

RESULT: Firewall rules were configured successfully: ports 80/443, the website IP and the program were all blocked as observed.

PRACTICAL NO. 8 : SSL/TLS CLIENT-SERVER

AIM: To set up an encrypted client-server channel using SSL/TLS sockets and observe the negotiated TLS version.

THEORY: The TLS handshake negotiates a protocol version and cipher suite, the server presents its X.509 certificate, and both sides derive symmetric session keys — after that every byte on the wire is encrypted and authenticated. The server builds ssl.SSLContext(PROTOCOL_TLS_SERVER) and loads its cert chain (server.crt + server.key); the client wraps the socket with ssl.create_default_context() and conn.version() reports the negotiated version. The certificate here is self-signed, so the lab client sets verify_mode = ssl.CERT_NONE — never done in production, where certs are verified against a trusted CA.

PROGRAM:

Client Side:

```
import socket, ssl
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE
with socket.create_connection(("localhost", 5000)) as sock:
    with ctx.wrap_socket(sock, server_hostname="localhost") as conn:
        print("TLS Version:", conn.version())
        while True:
            msg = input("You: ")
            conn.sendall(msg.encode())
            print("Server:", conn.recv(1024).decode())
            if msg.upper() == "BYE":
                break
```

Server Side:

```
import socket, ssl
ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
ctx.load_cert_chain("server.crt", "server.key")
server = socket.socket()
server.bind(("localhost", 5000))
server.listen(1)
print("SSL/TLS Server running...")
client, addr = server.accept()
with ctx.wrap_socket(client, server_side=True) as conn:
    print("Secure connection:", addr)
    while True:
        msg = conn.recv(1024).decode()
        print("Client:", msg)
        conn.sendall(b"MSG Received")
        if msg.upper() == "BYE":
            break
```

RESULT: The client and server connected on localhost:5000 and negotiated TLSv1.3.

PRACTICAL NO. 9 : INTRUSION DETECTION SYSTEM (IDS) AND TEST

AIM: To build a Python-based Intrusion Detection System that detects port-scan behaviour and test it with a TCP connect scan.

THEORY: scapy's sniff() captures live packets matching the BPF filter "tcp" and passes each to detect(), which watches for bare SYN packets (TCP flags = "S") — the first packet of every TCP connection and therefore of every connect scan. When one source IP has touched 5 distinct destination ports, the IDS prints "[ALERT] Possible Port Scan detected". The test script connect-scans 127.0.0.1 ports 5000–5050 with socket.connect_ex(), completing the three-way handshake on open ports — exactly the behaviour the IDS flags. Real NIDS (Snort, Suricata) build signature and anomaly engines on the same idea.

PROGRAM:

IDS:

```
from scapy.all import sniff, IP, TCP
ports = {}

def detect(pkt):
    if pkt.haslayer(IP) and pkt.haslayer(TCP) and pkt[TCP].flags == "S":
        src, dport = pkt[IP].src, pkt[TCP].dport
        ports.setdefault(src, set()).add(dport)
        print(f"Connection attempt: {src} -> Port {dport}")
        if len(ports[src]) >= 5:
            print(f"\n[ALERT] Possible Port Scan detected from {src}\n")
            ports[src].clear()

print("PYTHON BASED IDS - Monitoring network interfaces...")
sniff(filter="tcp", prn=detect, store=False)
```

Test:

```
import socket
print("Starting port scan test...\n")
for port in range(5000, 5051):
    s = socket.socket()
    s.settimeout(0.1)
    r = s.connect_ex(("127.0.0.1", port))
    print(f"Port {port} is {'OPEN' if r == 0 else 'CLOSED'}")
    s.close()
print("\nPort testing completed.")
```

RESULT: The test scan found port 5007 OPEN and all others CLOSED on 127.0.0.1; the IDS flags this behaviour once the 5-port threshold is crossed.